

Big Data Technology Overview

CII3F6 BIG DATA ANALYTICS

Program Studi Informatika

Fakultas Informatika – Telkom University



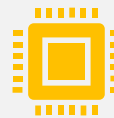
Big Data Infrastructure



Enabling Big Data Technology



Big Data Technology enabling Big Data
Operation & Big Data Analytics

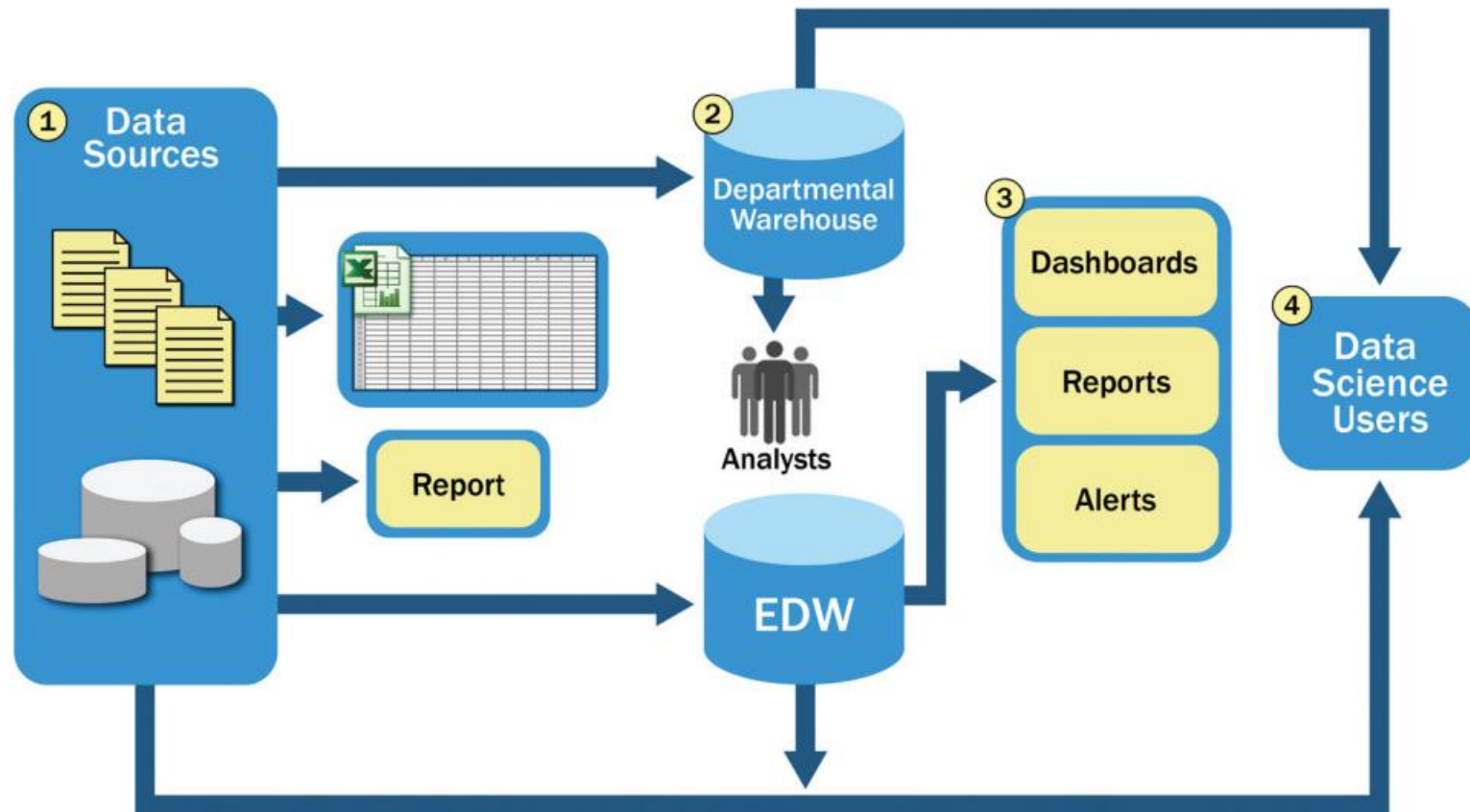


Big Data Operation Example : query, ETL,
crawling, etc

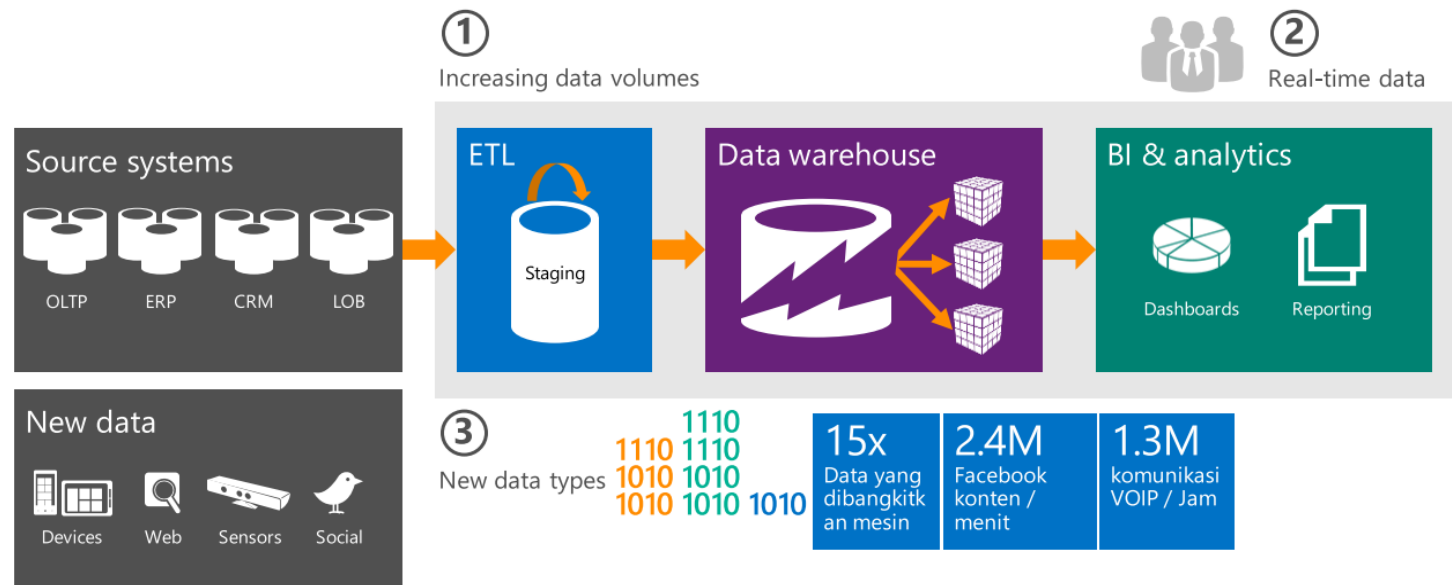
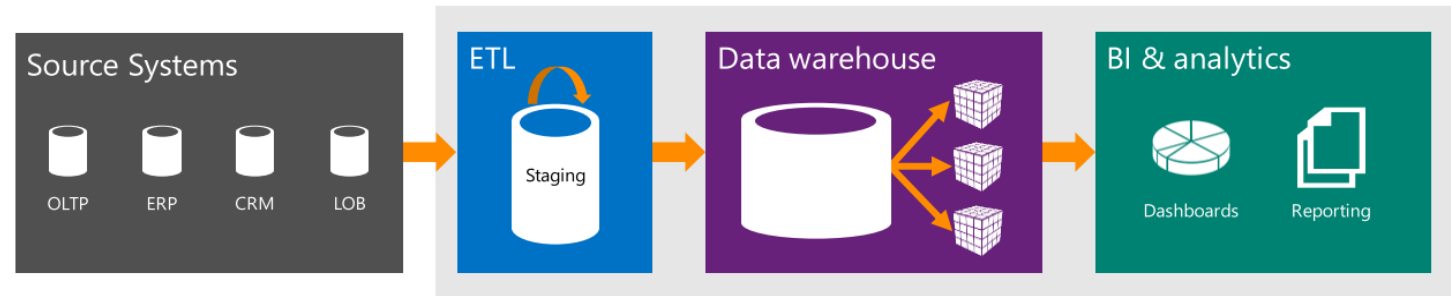


Big Data Analytics Example:
visualization, mining, statistic, etc

Traditional Solution



Data Evolution & Traditional Solution Challenge

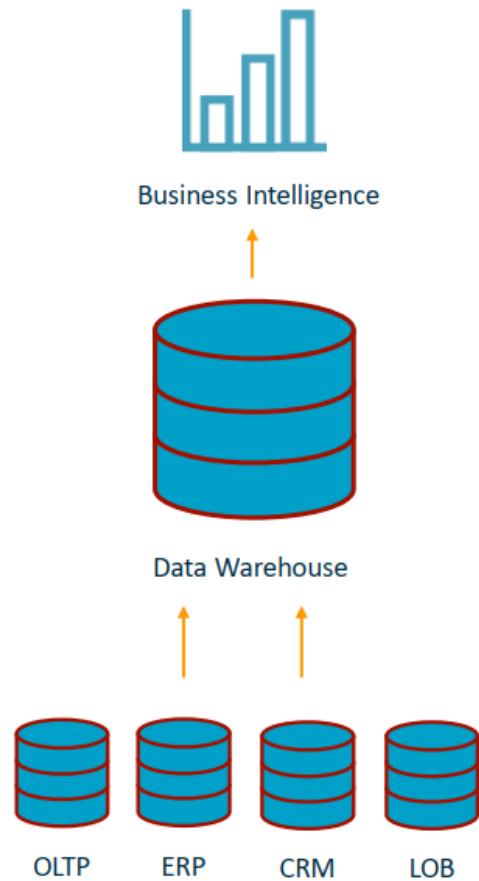


Data Lake (Big Data Solution)

- A **centralized repository** for both **structured** and **unstructured data**
- Store data as-is in open-source file formats to **enable direct analytics**
 - Enable advanced analytics across all of your data sources
- Data Lake is **architecture**, Hadoop is **Technology**
- **Decouple storage from compute**, allowing you to scale
- **Reduce complexity** in ETL and operational overhead



Traditional Data Analytics



Relational Data

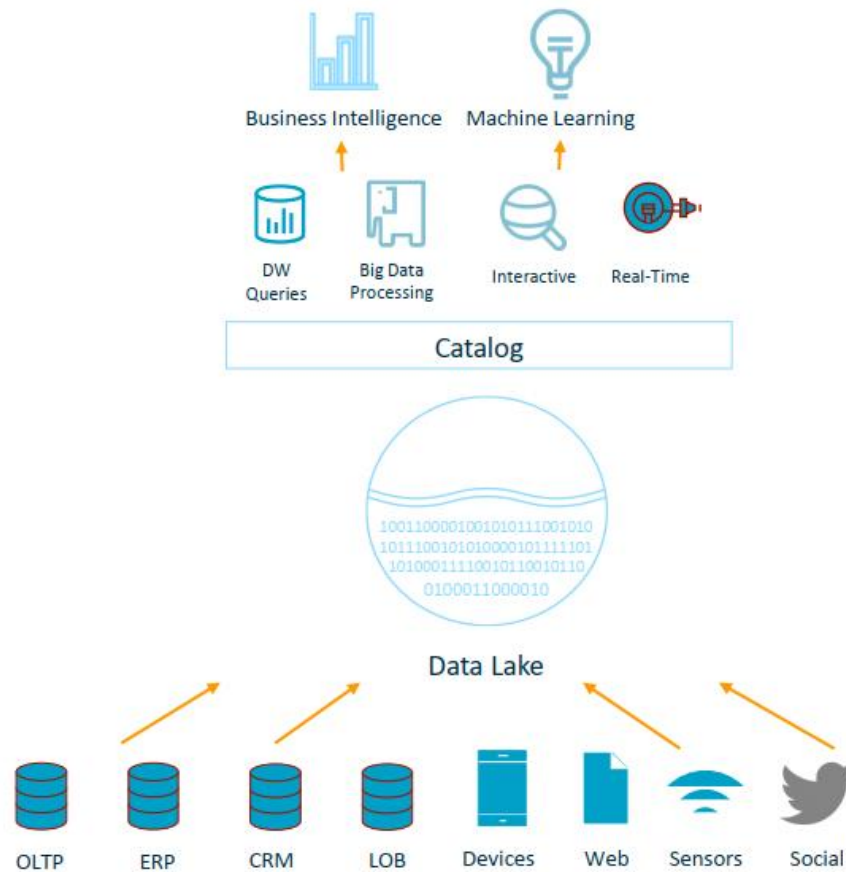
TBs-PBs Scale

Schema Defined Prior to Data Load

Operational and Ad Hoc Reporting

Large Initial Capex + \$\$K / TB/ Year

Big Data Analytics with Data Lake Approach



TB-EBs Scale

All Data in one place, a Single Source of Truth

Relational and Non-Relational Data

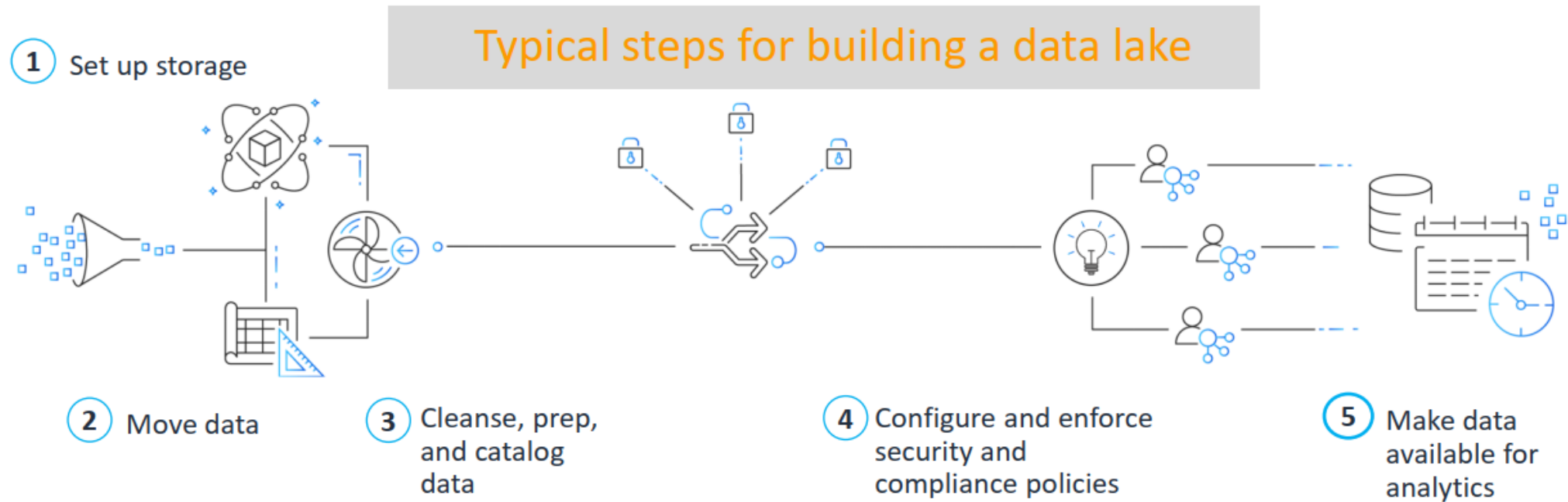
Decouples (low cost) Storage and Compute

Schema on Read

Diverse Analytical Engines

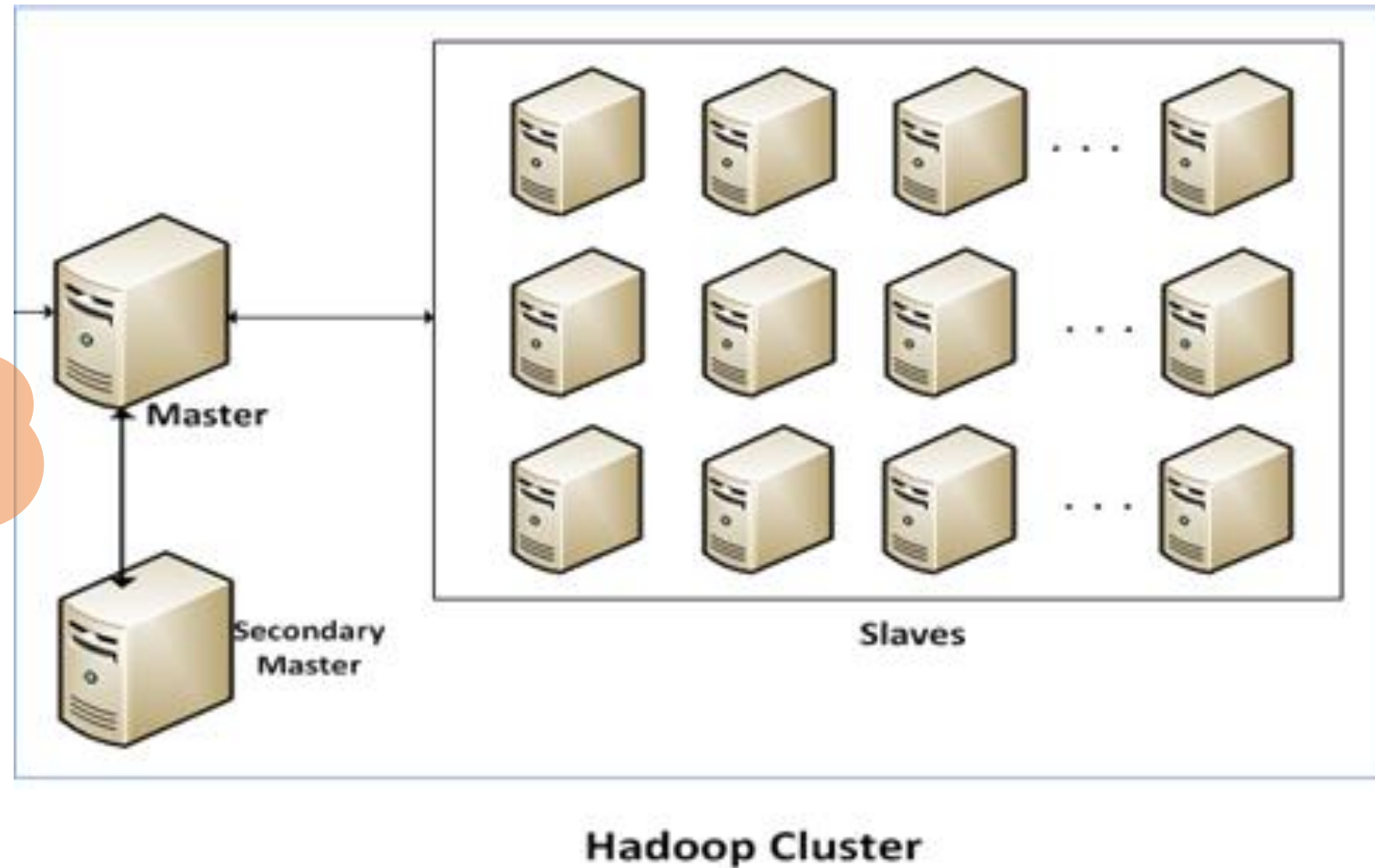
Data Lake Implementation

Implementing a Data Lake architecture requires a **broad set of tools and technologies** to serve an increasingly **diverse set of applications and use cases**.

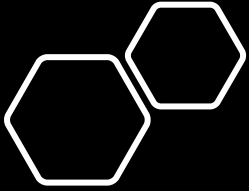


Hadoop Cluster

Master/Slave
Architecture



Merupakan himpunan komputer yang saling terkoneksi dan bekerja sama sedemikian sehingga terlihat seperti sebuah sistem tunggal.



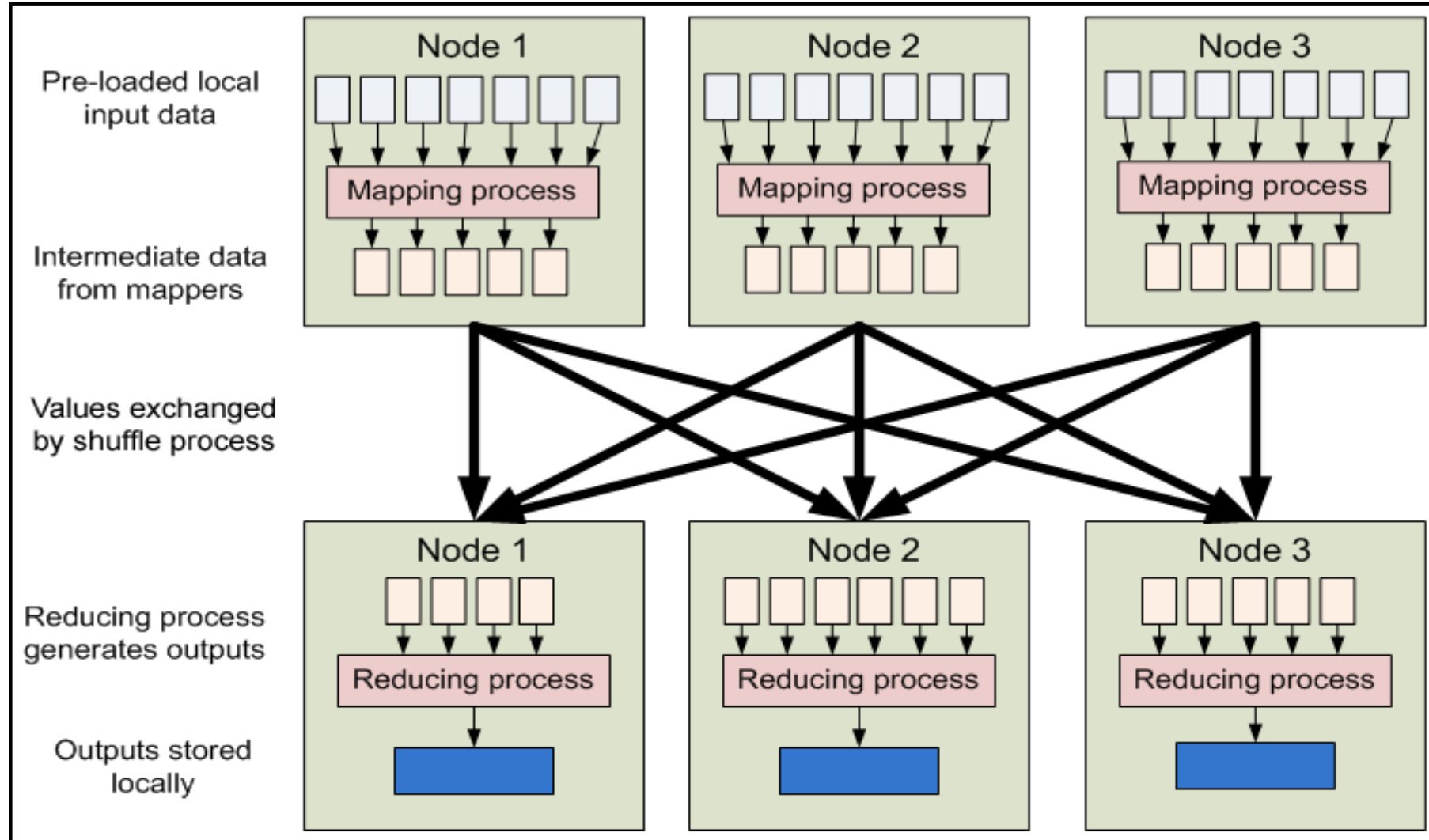
Map Reduce

- **MapReduce** is a programming model for efficient distributed computing
- Hadoop can run MapReduce programs written in various languages (Java, Ruby, Python, etc.)
- It works like a Unix pipeline
 - `cat input | grep | sort | uniq -c | cat > output`
 - **Input | Map | Shuffle & Sort | Reduce | Output**
- Efficiency from streaming through data, reducing seeks; and pipelining
- A good fit for a lot of applications: log processing, web index building

Map Reduce Feature

- Fine grained **Map** and **Reduce** tasks
 - Improved load balancing
 - Faster recovery from failed tasks
- Automatic **re-execution** on failure
 - In a large cluster, some nodes are always slow or flaky
 - Framework re-executes failed tasks
- **Locality optimizations**
 - With large data, bandwidth to data is a problem
 - Map-Reduce + HDFS is a very effective solution
 - Map-Reduce queries HDFS for locations of input data
 - Map tasks are scheduled close to the inputs when possible

MapReduce Flow



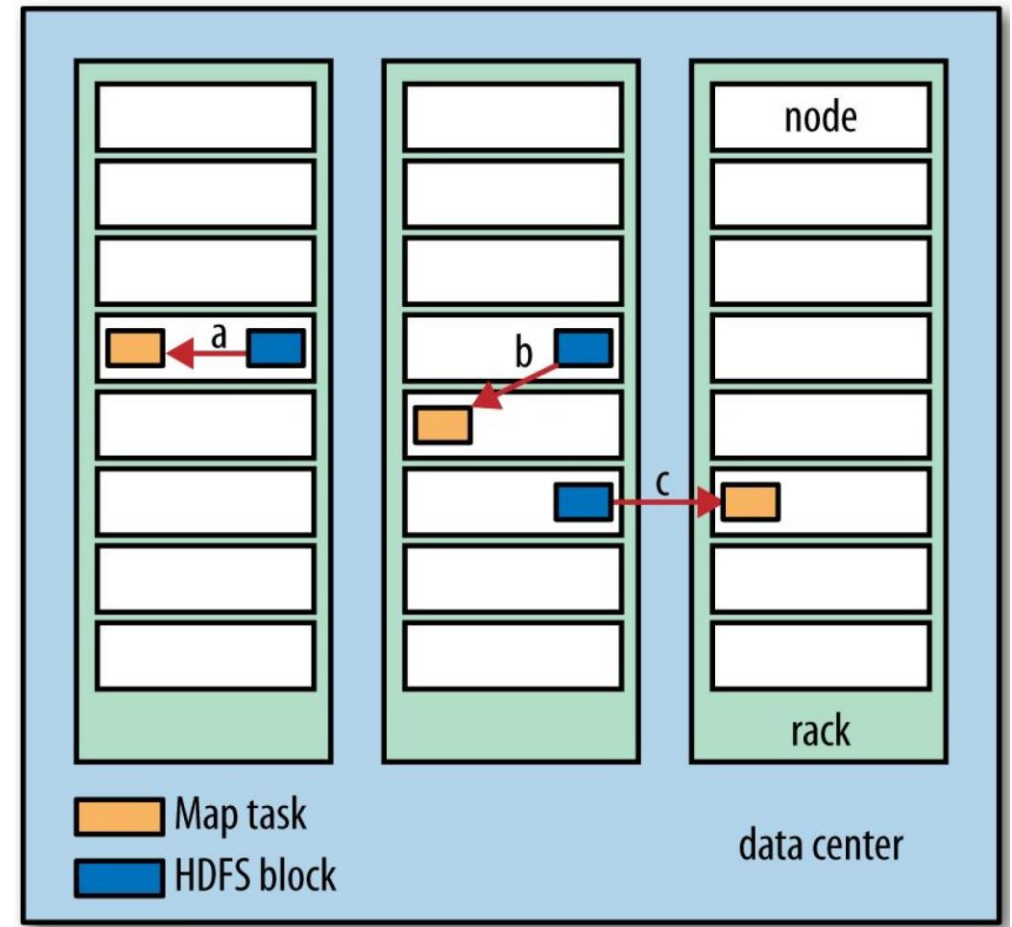
Map Tasks

- **Map** tasks write their output to the local disk, to be processed by **reduce** tasks to produce the final output.

(a) Data-local

(b) Rack-local

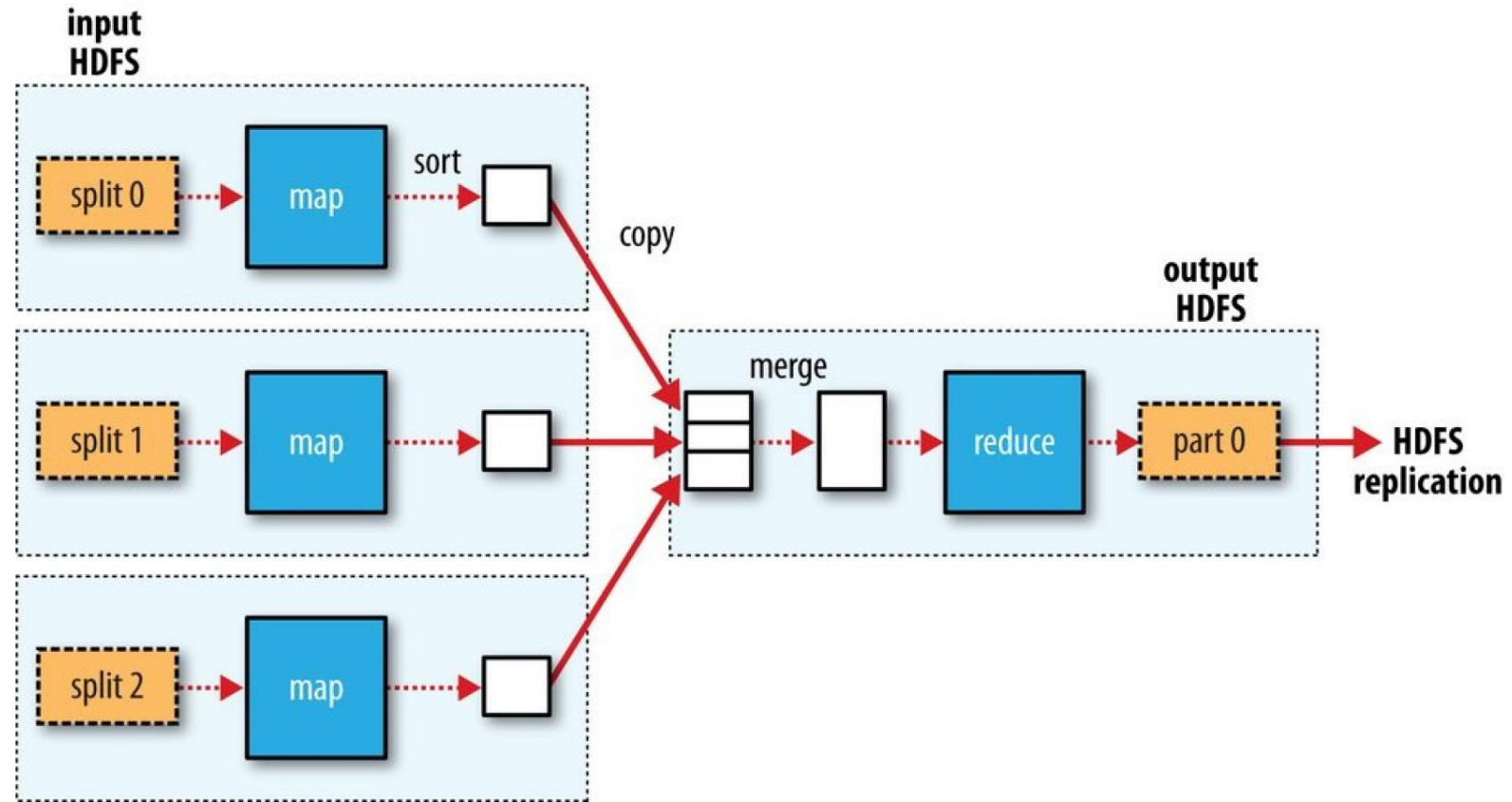
(c) Off-rack



Reduce Task

- **Reduce** tasks don't have the advantage of data locality
 - The input to a single reduce task is normally the output from all mappers.
- Therefore, the map outputs have to be **transferred** across the **network** to the node where the reduce task is running, where they are merged and then passed to the user-defined reduce function.
- The output of the reduce is normally **stored in HDFS** for reliability.
- For each HDFS block of the reduce output, the first replica is stored on the **local** node, with other replicas being stored on **off-rack nodes** for reliability.
 - Does not consume network bandwidth.

MapReduce data flow with a single reduce task



The Default MapReduce Job

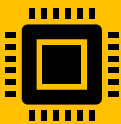


The number of reduce tasks is not governed by the size of the input, but instead is specified independently.

This can be seen in the **default MapReduce job**.

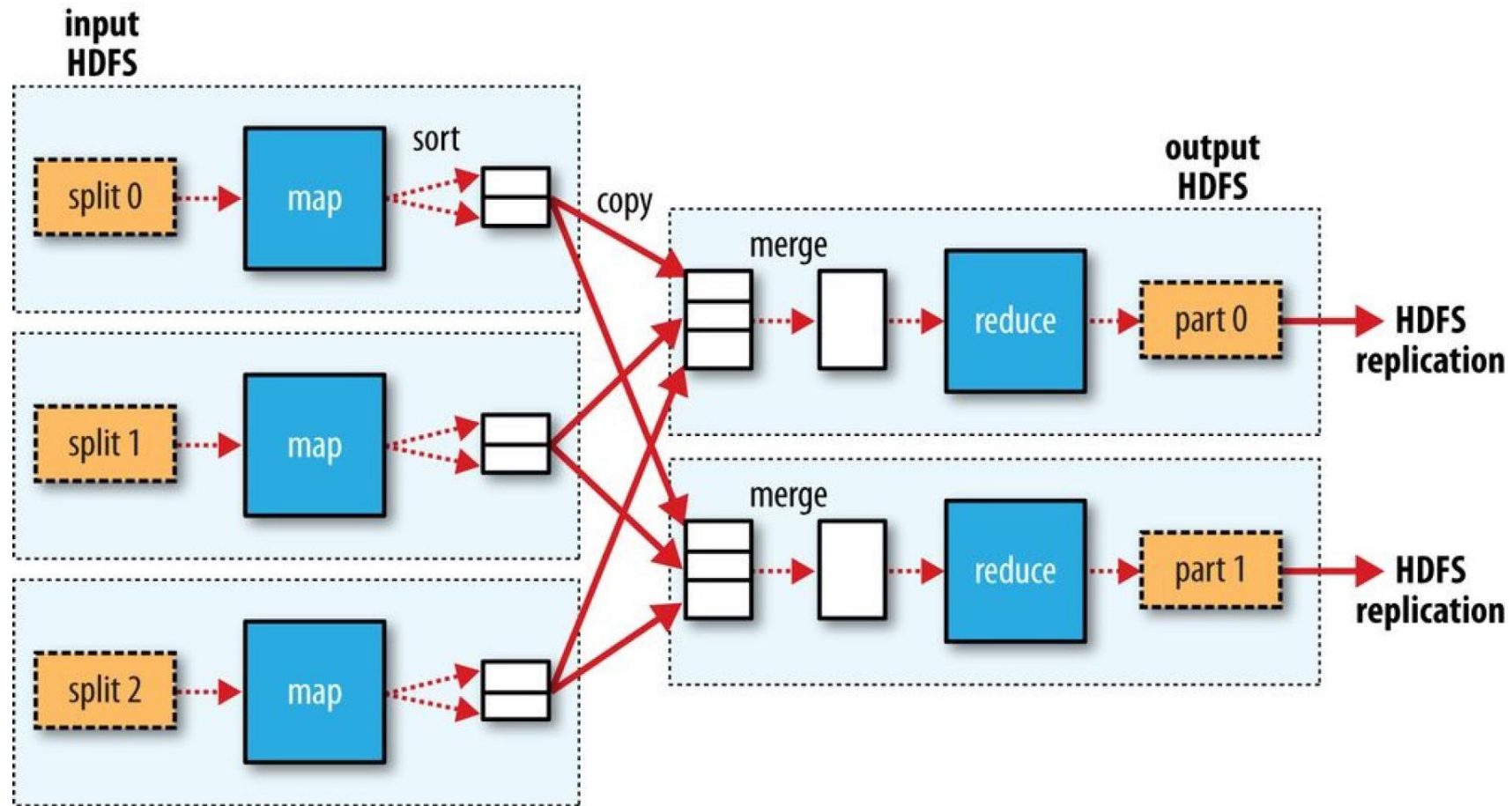


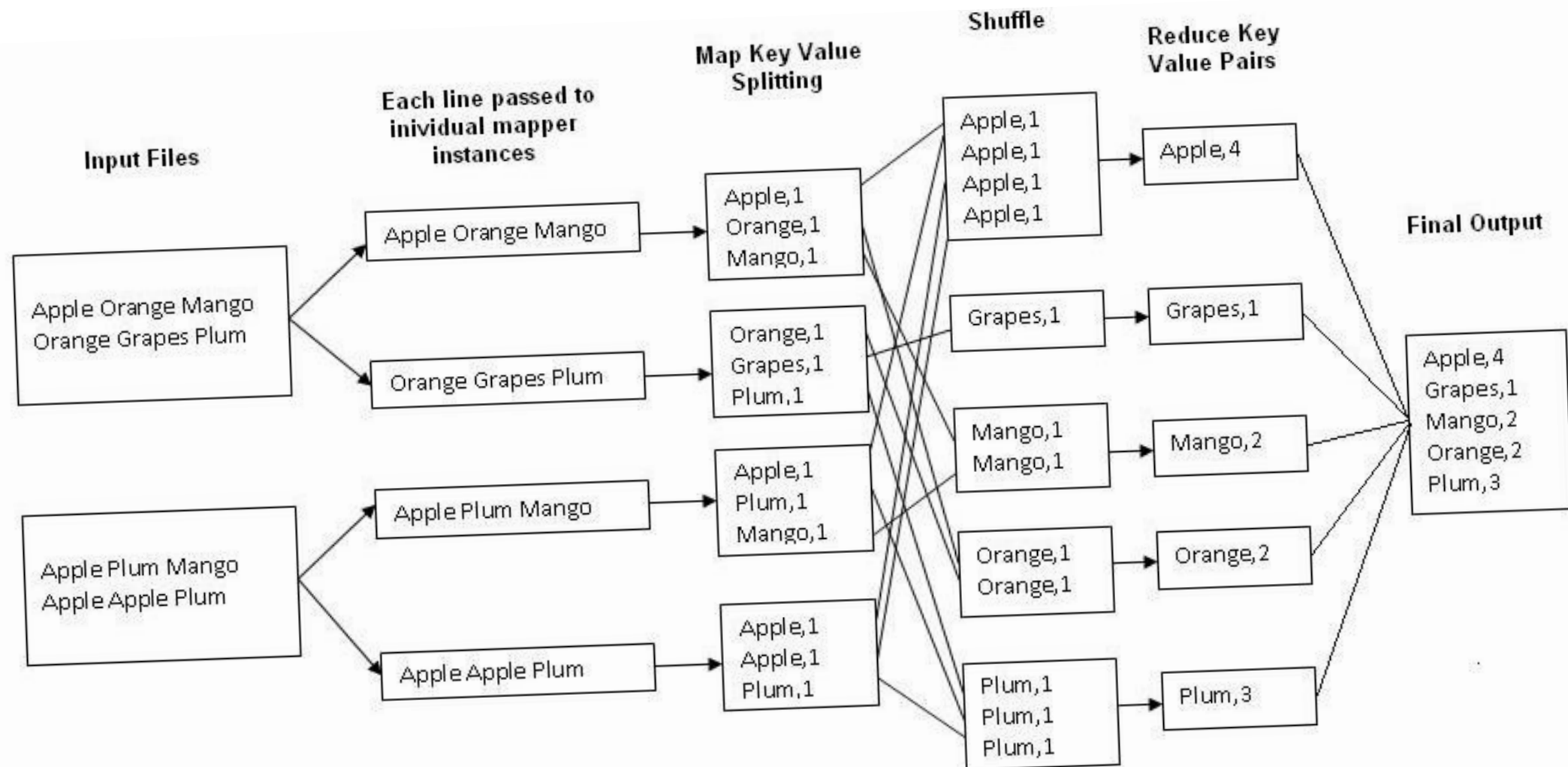
When there are **multiple reducers**, the map tasks partition their output, each creating one partition for each reduce task.



The data flow between map and reduce tasks is known as “**the shuffle**”, as each reduce task is fed by many map tasks.

MapReduce data flow with multiple reduce tasks





Popular Solution on Cloud Computing



Hadoop Cluster at Yahoo! (Credit: Yahoo)





Big Data Key Elements

- Data Acquisition and its protocol
- Data Management & Integration
- MapReduce Concept
- Big Data Infrastructure → Hadoop Framework, On premise, Cloud Computing
- Big Data Analytics -> Machine Learning & Data Mining



Thank you

Imelda Atastina

imelda@telkomuniversity.ac.id